

Implementasi IOT Dasar 3

©A. Haidar, Zen S Hadi, train4best 2023

POLBAN

pens

Train4best

1

Agenda

- Pulse Width Modulation
- Implementasi PWM menggunakan Buzzer
- Over The Air (OTA)
- Alur proses update melalui OTA
- TOPOLOGI – KONEKSI LOKAL
- Penerapan OTA
- Penerapan OTA melalui WEB

2

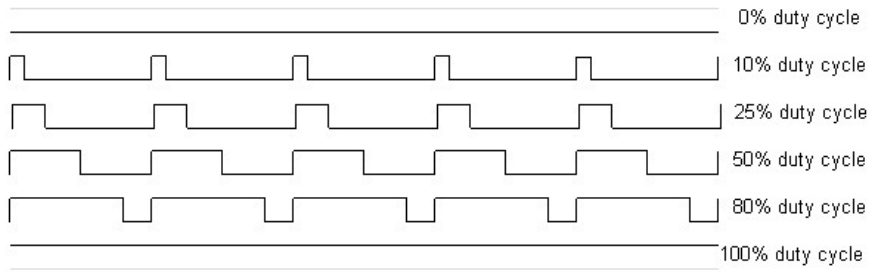
Pulse Width Modulation

- Modulasi Lebar Pulsa, atau PWM, adalah teknik untuk mendapatkan hasil analog dengan cara digital.
- Kontrol digital digunakan untuk membuat gelombang persegi, dengan cara sinyal diaktifkan dan dimatikan.
- Pola "on-off" ini dapat mensimulasikan tegangan *HIGH* pada papan (misalnya 3,3 V pada jenis WeMos ESP-8266) dan *LOW* (0 Volt) dengan mengubah porsi waktu yang digunakan sinyal untuk keadaan ON serta kondisi sinyal saat berada dalam keadaan OFF.

Pulse Width Modulation (cont.)

- Durasi antara waktu ON dan OFF tersebut dinamakan lebar pulsa.
- Untuk mendapatkan nilai analog yang bervariasi, hal yang perlu diubah adalah melakukan perubahan pada lebar pulsa tersebut.
- Proses mengulangi pola ON dan OFF ini jika dilakukan dengan cukup cepat, misalkan pada implementasi LED maka hasilnya seolah-olah sinyalnya adalah tegangan stabil antara 0 dan 3.3 volt yang mengendalikan kecerahan LED.

Pulse Width Modulation (cont.)



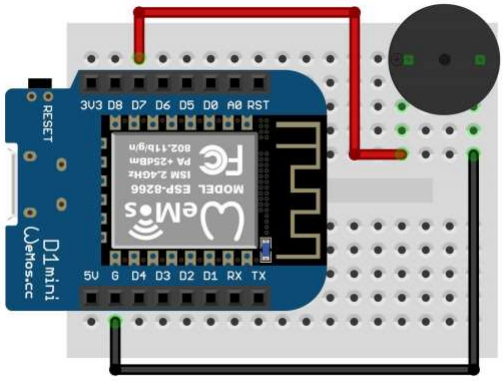
Sumber: https://docs.arduino.cc/tutorials/generic/secrets-of-arduino-pwm

- Hal ini juga dapat diimplementasikan pada suatu Buzzer sehingga menghasilkan perubahan suara berupa perubahan tinggi rendahnya suatu nada (pitch-control).

Implementasi PWM menggunakan Buzzer

```
const int pitchLow = 200;
const int pitchHigh = 1000;
int pitchStep = 1;
int currentPitch;
int delayTime;
const int speakerPin = D7;

void setup() {
  currentPitch = pitchLow;
  delayTime = 1;
  Serial.begin(115200);
  Serial.println("mulai");
}
```



Implementasi PWM menggunakan Buzzer (cont.)

```
void loop() {
  // meningkatkan frekuensi buzzer
  for(int currentPitch = pitchLow; currentPitch <= pitchHigh; currentPitch++){
    tone(speakerPin, currentPitch, 10);
    delay(delayTime);
    Serial.println("naik");
  }
  // menurunkan frekuensi buzzer
    for(int currentPitch = pitchHigh; currentPitch >= pitchLow; currentPitch--){
    tone(speakerPin, currentPitch, 10);
    delay(delayTime);
    Serial.println("turun");
  }
}
```


Over The Air (OTA)

Latar belakang kebutuhan *OTA:*

- terdapat bug pada perangkat yang telah dirancang, ingin melakukan update program
- proses upload terkoneksi melalui kabel
- Perangkat IoT yang telah terpasang sulit dijangkau dan dilakukan secara manual (kesulitan meningkat jika jumlah perangkat cukup banyak)

Over The Air (OTA – cont.)

Solusi - *OTA:*

- Implementasi OTA secara umum misalnya pada aplikasi *operating system* pada *mobile device (handphone)*, perubahan aplikasi dapat di-update secara remote

Over The Air (OTA – cont.)

- OTA, over the air adalah proses untuk memperbarui firmware dari ESP32 melalui koneksi nirkabel (seperti Wi-Fi), hal ini tentunya berfungsi sangat berbeda dengan metode biasanya yang digunakan melalui komunikasi serial, yaitu kabel.

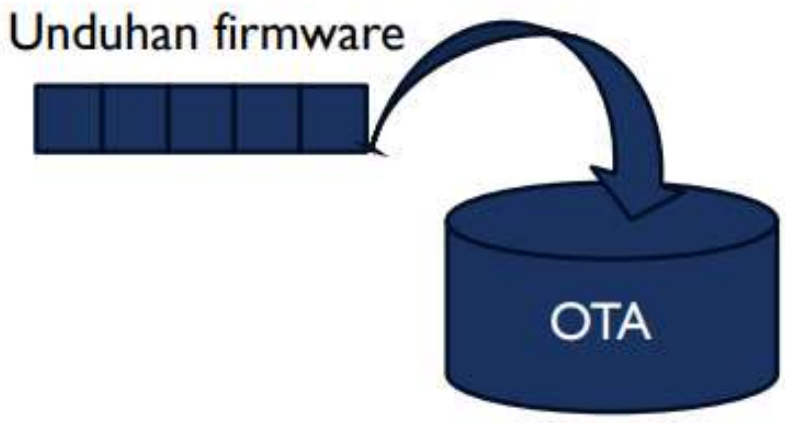
Alur proses update melalui OTA

1. Perangkat mengunduh firmware terbaru :
   a) Firmware yang diunduh disimpan pada ruang memori terpisah, misalnya kita anggap sebagai “ruang OTA” dan kode yang masih digunakan disimpan pada lokasi, misalnya kita anggap sebagai “ruang aplikasi”

Alur proses update melalui OTA (cont.)

1. Perangkat mengunduh firmware terbaru :
   b) Setelah seluruh firmware baru terunduh dan terverifikasi, bootloader akan bekerja dalam proses *update* (Bootloader berperan untuk menjalankan kode yang terdapat pada ruang aplikasi setiap kali perangkat melakukan *restart*.)

Alur proses update melalui OTA (cont.)



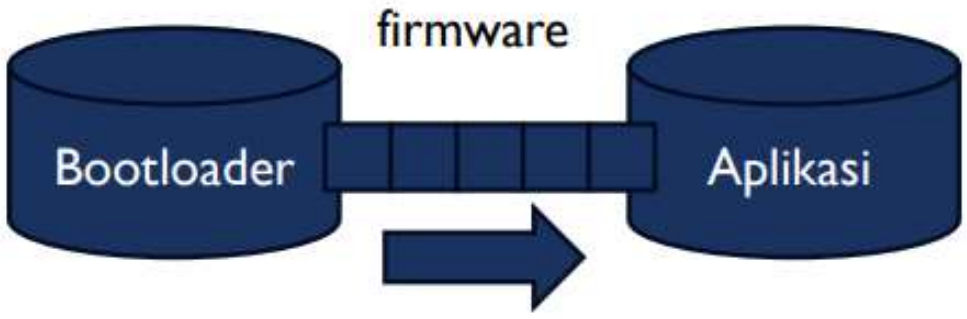
Alur proses update melalui OTA

2. Setiap kali program melakukan restart, kode pada bootloader akan dijalankan terlebih dahulu:
    a) Umumnya hanya memberi kode yang sudah ada, namun jika ada *firmware* terbaru, *bootloader* akan melihat ada kode aplikasi yang lebih baru dan tersedia setelah dilakukan *restart*

Alur proses update melalui OTA

2. Setiap kali program melakukan restart, kode pada bootloader akan dijalankan terlebih dahulu:
   b) Kode yang baru akan digunakan dan diberikan ke “ruang aplikasi”, maka firmware terbaru yang akan digunakan

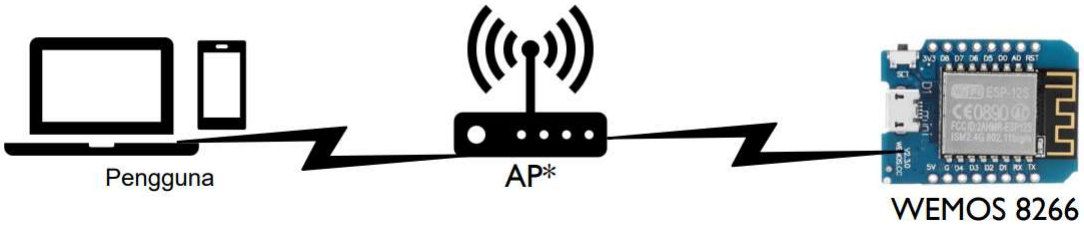
Alur proses update melalui OTA (cont.)



Alur proses update melalui OTA (cont.)

2. Bootloader juga dapat melakukan *flash code* untuk *factory mode* jika kode aplikasi *corupted*/rusak:
    c) Umumnya jika kode OTA dan kode *factory mode* disimpan pada penyimpanan eksternal jika ruang penyimpanan tidak cukup

TOPOLOGI – KONEKSI LOKAL



- Cek alamat *IP device*
- Buka alamat *IP* melalui *browser*

KONEKSI WLAN (WI-FI)

```
#include <ESP8266WiFi.h>
// SSID dan password Wi-Fi
const char* ssid = "Train4best";
const char* password = "train4best.com";
WiFiClient client;
String newHostname = "Train4best ESP8266";
```


KONEKSI WLAN (WI-FI – cont.)

```
void initWiFi() {
  WiFi.mode(WIFI_STA);// set mode station
  WiFi.hostname(newHostname.c_str()); //set nama
  WiFi.begin(ssid, password); //connect AP
  Serial.print("Connecting to WiFi ..");
  while (WiFi.status() != WL_CONNECTED) {
    Serial.print('.');
    delay(1000);
  }
  Serial.println(WiFi.localIP());
  // auto connect
  // WiFi.setAutoReconnect(true);
  // WiFi.persistent(true);
}
```

KONEKSI WLAN (WI-FI – cont.)

```
void setup() {
  Serial.begin(115200);
  initWiFi();
  Serial.print("RSSI: ");
  Serial.println(WiFi.RSSI());
  Serial.println("\nWiFi connected\n");
}
void loop() {
  while (WiFi.status() != WL_CONNECTED) {
    Serial.print("...");
    delay(1000);
  }
}
```


Penerapan OTA

- Pastikan *device* yang digunakan tepat dan pilih LOLIN (WEMOS) D1 R2 & mini
- Buka File > Example > ArduinoOTA>BasicOTA
- Nyalakan *private hotspot* (simpan password dan nama SSID Wi-Fi)
- Program (sesuaikan SSID dan password)
- Program OTA harus selalu ada dalam program yang digunakan agar OTA selalu berfungsi



Penerapan OTA (cont.)

```
//library yang digunakan
#include <ESP8266WiFi.h>
#include <ArduinoOTA.h>
// Nama dan password dari AP Wi-Fi
const char* ssid = "Train4best";
const char* password = "train4best.com";
String newHostname = "Train4best ESP8266";

//inisialisasi nomor pinout, misal LED builtin
const int ESP_BUILTIN_LED = 2;
```


Penerapan OTA (cont.)

```
void setup() {
  //mulai mode komunikasi serial
  Serial.begin(115200);
  //Serial.println("Booting");
  //WiFi.mode(WIFI_STA);
  WiFi.hostname(newHostname.c_str()); //set nama
  WiFi.begin(ssid, password); //mulai wi-fi
  while (WiFi.waitForConnectResult() != WL_CONNECTED) {
    Serial.print('.');
    delay(1000);
    //Serial.println("Connection Failed! Rebooting...");
    //delay(5000);
    //ESP.restart();
  }
```

Penerapan OTA (cont.)

```
// Port defaults to 8266
//ArduinoOTA.setPort(8266);
// Hostname defaults to esp8266-[ChipID]
// ArduinoOTA.setHostname("myesp8266");
// No authentication by default
// ArduinoOTA.setPassword((const char *)"123");
ArduinoOTA.onStart([]() {
  Serial.println("Start");
});
ArduinoOTA.onEnd([]() {
  Serial.println("\nEnd");
});
ArduinoOTA.onProgress([](unsigned int progress, unsigned int total) {
  Serial.printf("Progress: %u%%\r", (progress / (total / 100)));
});
```


Penerapan OTA (cont.)

```
ArduinoOTA.onError([](ota_error_t error) {
    Serial.printf("Error[%u]: ", error);
    if (error == OTA_AUTH_ERROR) Serial.println("Auth Failed");
    else if (error == OTA_BEGIN_ERROR) Serial.println("Begin Failed");
    else if (error == OTA_CONNECT_ERROR) Serial.println("Connect Failed");
    else if (error == OTA_RECEIVE_ERROR) Serial.println("Receive Failed");
    else if (error == OTA_END_ERROR) Serial.println("End Failed");
  });
  ArduinoOTA.begin();
  Serial.println("Ready");
  Serial.print("IP address: ");
  Serial.println(WiFi.localIP());
  Serial.print("RSSI: ");
  Serial.println(WiFi.RSSI());
  Serial.println("\nWiFi connected\n");
```

Penerapan OTA (cont.)

```
//inisialisasi pinout
  pinMode(ESP_BUILTIN_LED, OUTPUT);
}

void loop() {
  ArduinoOTA.handle();
  while (WiFi.status() != WL_CONNECTED) {
    Serial.print("...");
    delay(1000);
  }
  //isi code loop anda di sini
  digitalWrite(ESP_BUILTIN_LED, LOW);
  delay(1000);
  digitalWrite(ESP_BUILTIN_LED, HIGH);
  delay(1000);
}
```


Penerapan OTA melalui WEB

```
#include <ESP8266WiFi.h>
#include <WiFiClient.h>
#include <ESP8266WebServer.h>
#include <ESP8266mDNS.h>

#ifndef STASSID
#define STASSID "Train4best"
#define STAPSK "train4best.com"
//#define STASSID2 "Train4bestesp"
//#define STAPSK2 "train4best.com"
#endif
```

Penerapan OTA melalui WEB (cont.)

```
const int ESP_BUILTIN_LED = 2;
const char* host = "train4best";
const char* ssid = STASSID;
const char* password = STAPSK;
//const char* ssid2 = STASSID2;
//const char* password2 = STAPSK2;
ESP8266WebServer server(80);
const char* serverIndex = "<form method='POST'
action='/update' enctype='multipart/form-data'><input
type='file' name='update'><input type='submit'
value='Update'></form>";
```


Penerapan OTA melalui WEB (cont.)

```
void setup(void) {
  Serial.begin(115200);
  Serial.println();
  Serial.println("Booting Sketch...");
  WiFi.mode(WIFI_STA);//WiFi.mode(WIFI_AP_STA);//mode ap bisa untuk koneksi langsung ke board
//  WiFi.softAP(ssid2, password2);
  WiFi.begin(ssid, password);
  if (WiFi.waitForConnectResult() == WL_CONNECTED) {
    MDNS.begin(host);
    server.on("/", HTTP_GET, []() {
      server.sendHeader("Connection", "close");
      server.send(200, "text/html", serverIndex);
    });
```

Penerapan OTA melalui WEB (cont.)

```
server.on(
  "/update", HTTP_POST, []() {
    server.sendHeader("Connection", "close");
    server.send(200, "text/plain", (Update.hasError()) ? "FAIL" : "OK");
    ESP.restart();
  },
  []() {
    HTTPUpload& upload = server.upload();
    if (upload.status == UPLOAD_FILE_START) {
      Serial.setDebugOutput(true);
      WiFiUDP::stopAll();
      Serial.printf("Update: %s\n", upload.filename.c_str());
      uint32_t maxSketchSpace = (ESP.getFreeSketchSpace() - 0x1000) & 0xFFFFF000;
```


Penerapan OTA melalui WEB (cont.)

```
  if (!Update.begin(maxSketchSpace)) {  // start with max available size
      Update.printError(Serial);
    }
  } else if (upload.status == UPLOAD_FILE_WRITE) {
    if (Update.write(upload.buf, upload.currentSize) != upload.currentSize) {
      Update.printError(Serial);
    }
  } else if (upload.status == UPLOAD_FILE_END) {
    if (Update.end(true)) {  // true to set the size to the current progress
      Serial.printf("Update Success: %u\nRebooting...\n", upload.totalSize);
    } else {
      Update.printError(Serial);
    }
    Serial.setDebugOutput(false);
  }
```

Penerapan OTA melalui WEB (cont.)

```
        yield();
      });
    server.begin();
    MDNS.addService("http", "tcp", 80);

    Serial.printf("Ready! Open http://%s in your browser\n", host);
  } else {
    Serial.println("WiFi Failed");
  }
    pinMode(ESP_BUILTIN_LED, OUTPUT);
}
void loop(void) {
  server.handleClient();
  MDNS.update();
    //isi code loop anda di sini
  digitalWrite(ESP_BUILTIN_LED, LOW);
  delay(500);
  digitalWrite(ESP_BUILTIN_LED, HIGH);
  delay(500);
}
```


Latihan

- Buatlah eksperimen dengan OTA dengan perubahan delay 150 ms

Good Luck….!